Lists in Python

A **list** is a built-in data structure in Python used to store multiple values in a single variable. Below is a simple point-wise explanation:

1. Defining a List

- A **list** is an ordered collection of elements.
- Lists can store **different data types** (integers, strings, floats, etc.).
- Lists are **mutable**, meaning elements can be changed.
- Example:

```
my_list = [10, "apple", 3.5, True]
```

2. Creating a List

- Lists are created using **square brackets [ ]**.
- Example of an empty and non-empty list:

```
empty_list = []  # Empty list

fruits = ["Apple", "Banana", "Cherry"]  # List with elements
```

3. Accessing Values from a List

- Elements in a list are accessed using **indexing**.
- Indexing starts from **0** (zero-based indexing).
- Example:

```
fruits = ["Apple", "Banana", "Cherry"]

print(fruits[0])  # Output: Apple

print(fruits[1])  # Output: Banana
```

4. Updating Elements of a List

- Lists are **mutable**, meaning we can change elements.
- Example:

```
numbers = [10, 20, 30, 40]

numbers[1] = 25  # Changing second element

print(numbers)  # Output: [10, 25, 30, 40]
```

5. Concatenation of Two Lists

- Two lists can be combined using the **+ operator**.
- Example:

```
list1 = [1, 2, 3]
```

```
list2 = [4, 5, 6]

result = list1 + list2

print(result)  # Output: [1, 2, 3, 4, 5, 6]
```

Conclusion:

- **Lists store multiple values and are mutable.**
- **Elements are accessed using indexing (0 for the first element).**
- **We can update, modify, and concatenate lists easily.**

Practical programs

---

1. Program to Find the Maximum and Minimum Elements in a List

Definition:

Write a Python program that accepts a list of numbers from the user and finds both the maximum and minimum elements present in the list.

```
# Input list from user

numbers = list(map(int, input("Enter numbers separated by space: ").split()))


minvalue = min(numbers)

maxvalue = max(numbers)


print("Minimum from list = {}, Maximum from list = {}".format(minvalue,maxvalue))

# Displaying results

print(f"Maximum element: {max_value}")

print(f"Minimum element: {min_value}")
```

---

2. Program to Remove Duplicates from a List

Definition:

Write a Python program that takes a list of numbers from the user and removes all duplicate elements, preserving only unique values.

```
# Input list

numbers = [1, 2, 3, 2, 4, 5, 6, 4, 7, 8, 1, 9, 3]
```

```
# Empty list to store duplicates
duplicates = []
# Check for duplicates
for i in range(len(numbers)):
    for j in range(i + 1, len(numbers)):
        if numbers[i] == numbers[j] and numbers[i] not in duplicates:
            duplicates.append(numbers[i])

# Print the duplicate values
print("Duplicate values:", duplicates)
```

Explanation:

1. The program takes a list of numbers.
2. It initializes an empty list duplicates to store duplicate values.
3. It uses nested loops to compare each element with the remaining elements.
4. If a duplicate is found and is not already in duplicates, it is added.
5. Finally, it prints the list of duplicate values.

Sample Output:

Duplicate values: [1, 2, 3, 4]

---

3. Program to Merge Two Lists and Sort the Result

Definition:

Write a Python program that accepts two lists from the user, merges them into a single list, and sorts the final list in ascending order.

```
#Input two lists from user
list1 = list(map(int, input("Enter elements of first list: ").split()))
list2 = list(map(int, input("Enter elements of second list: ").split()))
# Merging and sorting
merged_list = list1 + list2
sorted_list = sorted(merged_list)
# Displaying result
```

```
print("Merged and sorted list:", sorted_list)
```

Output of the program

Enter elements of first list: 25 30 50 101 9

Enter elements of second list: 27 727 51 11 525

Merged and sorted list: [9, 11, 25, 27, 30, 50, 51, 101, 525, 727]

---

4. Program to find only duplicate (i.e. repeating) elements in a List

```
# Input list
numbers = [1, 2, 3, 2, 4, 5, 6, 4, 7, 8, 1, 9, 3]
# Empty list to store duplicates
duplicates = []
# Check for duplicates
for i in range(len(numbers)):
    for j in range(i + 1, len(numbers)):
        if numbers[i] == numbers[j] and numbers[i] not in duplicates:
            duplicates.append(numbers[i])

# Print the duplicate values
print("Duplicate values:", duplicates)
```

Explanation:

1. The program takes a list of numbers.
2. It initializes an empty list duplicates to store duplicate values.
3. It uses nested loops to compare each element with the remaining elements.
4. If a duplicate is found and is not already in duplicates, it is added.
5. Finally, it prints the list of duplicate values.

Sample Output:

Duplicate values: [1, 2, 3, 4]

---

5. Program to Rotate a List to the Right by a Given Number of Positions

```
# Input list from user
numbers = list(map(int, input("Enter numbers separated by space: ").split()))
```

```
# Input rotation count
n = int(input("Enter number of positions to rotate: "))

# Performing right rotation
n = n % len(numbers)  # Handling cases where n is larger than list size
rotated_list = numbers[-n:] + numbers[:-n]

# Displaying result
print("List after rotation:", rotated_list)
```

<u>Output of the program</u>

Enter numbers separated by space: 5 7 9 11 3

Enter number of positions to rotate: 2

List after rotation: [11, 3, 5, 7, 9]

Explanation of the Program: Rotate a List to the Right

1. **Input List from User**
    - The user enters numbers separated by spaces.
    - The input() function takes the user's input as a string.
    - split() breaks the string into individual number strings.
    - map(int, ...) converts each number string into an integer.
    - list(...) stores these integers in a list called numbers.
    - **Example Input:** 1 2 3 4 5
    - **Resulting List:** [1, 2, 3, 4, 5]
2. **Input Rotation Count**
    - The user enters how many positions to rotate the list to the right.
    - The int(input()) function converts this input into an integer n.
    - **Example Input:** 2
3. **Handling Large Rotation Values**

- If n is larger than the list size, rotating more than the length of the list gives the same result as rotating n % len(numbers).
- n = n % len(numbers) ensures n is within valid range.

  Example: If the list has 5 elements and n = 7, then n = 7 % 5 = 2.

4. **Performing Right Rotation**
   - The last n elements are extracted using numbers[-n:].
   - The remaining elements (excluding the last n) are extracted using numbers[:-n].
   - The two parts are combined:
   - rotated_list = numbers[-n:] + numbers[:-n]
   - **Example Calculation:**
     - numbers = [1, 2, 3, 4, 5]
     - n = 2
     - numbers[-2:] → [4, 5] (last 2 elements)
     - numbers[:-2] → [1, 2, 3] (all except last 2)
     - Combined: [4, 5] + [1, 2, 3] = [4, 5, 1, 2, 3]
5. **Displaying the Result**
   - The print() function outputs the rotated list.
   - **Example Output:**
   - List after rotation: [4, 5, 1, 2, 3]